

JavaScript Conditions

What is a Condition?

A **condition** is an expression that evaluates to either `true` or `false`.

Conditions allow JavaScript to make decisions and execute different blocks of code based on whether a condition is true or false.

Why Do We Use Conditions?

Conditions help a program:

- Make decisions
- Validate user input
- Check login status
- Control program flow
- Display different content

Types of Conditional Statements

1. if
2. if...else
3. else if
4. Nested if
5. switch
6. Ternary Operator

1. if Statement

Definition

The `if` statement executes a block of code only if the condition is true.

Syntax

```
if (condition) {  
    // code  
}
```

Example

```
let age = 20;  
  
if (age >= 18) {  
    console.log("You can vote.");  
}
```

Output

```
You can vote.
```

2. if...else Statement

Definition

The `if...else` statement executes one block if the condition is true and another block if it is false.

Syntax

```
if (condition) {  
    // true block  
} else {  
    // false block  
}
```

Example

```
let age = 15;  
  
if (age >= 18) {  
    console.log("Adult");  
} else {  
    console.log("Minor");  
}
```

Output

Minor

3. else if Statement

Definition

Use `else if` to check multiple conditions.

Syntax

```
if (condition1) {  
  
}  
else if (condition2) {  
  
}  
else {  
  
}
```

Example

```
let marks = 82;  
  
if (marks >= 90) {  
    console.log("Grade A+");  
}  
else if (marks >= 80) {  
    console.log("Grade A");  
}  
else if (marks >= 70) {  
    console.log("Grade B");  
}  
else {  
    console.log("Fail");  
}
```

Output

Grade A

4. Nested if Statement

Definition

A nested `if` is an `if` statement inside another `if` statement.

Example

```
let age = 22;
let hasID = true;

if (age >= 18) {

    if (hasID) {
        console.log("Entry Allowed");
    }

}
```

Output

```
Entry Allowed
```

5. switch Statement

Definition

The `switch` statement selects one of many code blocks based on a matching value.

Syntax

```
switch(value){

case value1:
```

```
        break;

    case value2:
        break;

    default:
    }
```

Example

```
let day = 3;

switch(day){

    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
        console.log("Invalid Day");

}
```

Output

Wednesday

6. Ternary Operator

Definition

The ternary operator is a shorter way to write an `if...else` statement.

Syntax

```
condition ? trueValue : falseValue;
```

Example

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

Output

```
Adult
```

Comparison Operators Used in Conditions

Operator	Description
==	Equal
===	Strict Equal
!=	Not Equal
!==	Strict Not Equal

Operator	Description
>	Greater Than
<	Less Than
>=	Greater Than or Equal
<=	Less Than or Equal

Logical Operators Used in Conditions

Operator	Description
&&	AND
	OR
!	NOT

Example

```
let age = 22;
let hasID = true;

if(age >=18 && hasID){
  console.log("Access Granted");
}
```

Real Project Examples

Login Check

```
let isLoggedIn = true;

if(isLoggedIn){
  console.log("Welcome Back");
}else{
```

```
    console.log("Please Login");  
}
```

Password Check

```
let password = "123456";  
  
if(password.length >= 6){  
    console.log("Strong Password");  
}else{  
    console.log("Weak Password");  
}
```

Shopping Discount

```
let total = 1500;  
  
if(total >= 1000){  
    console.log("Discount Applied");  
}else{  
    console.log("No Discount");  
}
```

Weather App

```
let temperature = 36;  
  
if(temperature > 35){  
    console.log("Very Hot");  
}else{  
    console.log("Normal Weather");  
}
```

Best Practices

Use `===` instead of `==`

```
if(age === 18){  
}
```

Use meaningful variable names.

```
let isLoggedIn = true;
```

Avoid deeply nested conditions.

Common Mistakes

Missing Curly Braces

```
if(age >=18)  
console.log("Adult");
```

```
if(age >=18){  
    console.log("Adult");  
}
```

Using = Instead of ===

```
if(age = 18){  
}
```

```
if(age === 18){  
}
```

Interview Questions

1. What is a conditional statement?

A conditional statement is used to execute different blocks of code based on whether a condition is true or false.

2. What is the difference between if and switch?

if	switch
Works with any condition	Works with one value
Better for ranges	Better for fixed values
More flexible	Easier to read with many cases

3. When should you use switch?

Use `switch` when comparing one variable against multiple fixed values.

Summary

Statement	Purpose
if	Execute code if condition is true
if...else	Two possible outcomes
else if	Multiple conditions
Nested if	Condition inside another condition
switch	Multiple fixed values

Statement	Purpose
Ternary	Short if...else